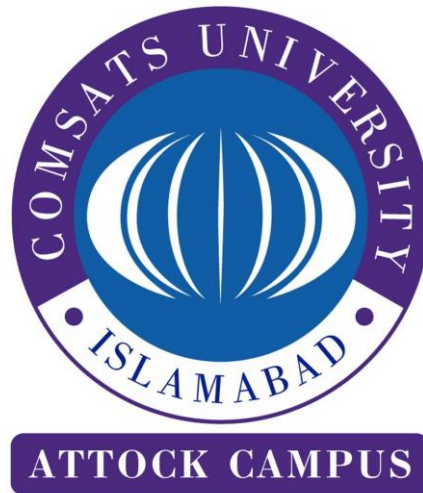


Data Structures Lab

Assignment 01



By

Mamoona Anwar

FA25-BCS-020

Submitted to,

Mr. Bilal Bukhari

October 04, 2026

Department of Computer Science
COMSATS University Islamabad, Attock Campus

Singly Linked List

Task 01: Display singly linked list in reverse order

```
#include <iostream>          // Include input-output library

using namespace std;        // Use standard namespace


struct Node {               // Define node structure

    int data;               // Store integer data

    Node* next;            // Pointer to next node

};


Node* createNode(int value) { // Function to create a new node

    Node* n = new Node;      // Allocate memory for node

    n->data = value;          // Set data value

    n->next = nullptr;        // Set next pointer to null

    return n;                // Return new node

}


void insertEnd(Node*& head, int value) { // Insert node at end

    Node* n = createNode(value); // Create new node

    if (head == nullptr) {       // If list is empty

        head = n;               // Make new node the head

        return;                 // Exit function

    }

    Node* temp = head;           // Start from head

    while (temp->next != nullptr) { // Traverse to last node

        temp = temp->next;       // Move to next node

    }

}
```

```

    }

    temp->next = n;          // Link last node to new node
}

void printReverse(Node* head) {    // Print list in reverse order

    if (head == nullptr) return;  // Base case: empty node

    printReverse(head->next);      // Recursively go to end

    cout << head->data << " ";    // Print while returning back
}

void printForward(Node* head) {    // Print list in normal order

    Node* temp = head;           // Start from head

    while (temp != nullptr) {     // Until end of list

        cout << temp->data << " "; // Print current data

        temp = temp->next;         // Move forward
    }

    cout << endl;                // Print newline
}

int main() {                      // Main function

    Node* head = nullptr;         // Initially empty list

    insertEnd(head, 10);           // Insert 10

    insertEnd(head, 20);           // Insert 20

    insertEnd(head, 30);           // Insert 30

    insertEnd(head, 40);           // Insert 40

    cout << "Forward: ";          // Print label

```

```

    printForward(head);        // Print normal list

    cout << "Reverse: ";      // Print label

    printReverse(head);        // Print reversed order

    cout << endl;             // Print newline

    return 0;                  // End program
}

```

Output:

```

Forward: 10 20 30 40
Reverse: 40 30 20 10

-----
Process exited after 0.144 seconds with return value 0
Press any key to continue . . .

```

Task 02: Reverse the singly Linked list itself

```

#include <iostream>            // Include input-output library

using namespace std;          // Use standard namespace


struct Node {                 // Define node structure

    int data;                  // Store data

    Node* next;                // Pointer to next node
};


Node* createNode(int value) {  // Create new node

    Node* n = new Node;        // Allocate memory

    n->data = value;            // Set data

    n->next = nullptr;          // Set next null

    return n;                  // Return node
}

```

```

void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) { // If empty list

        head = n; // Set head

        return; // Exit

    }

    Node* temp = head; // Start from head

    while (temp->next != nullptr) { // Go to last node

        temp = temp->next; // Move forward

    }

    temp->next = n; // Link last node

}

```

```

Node* reverseList(Node* head) { // Reverse linked list

    Node* prev = nullptr; // Previous pointer starts null

    Node* curr = head; // Current pointer starts head

    Node* next = nullptr; // Next pointer for saving

    while (curr != nullptr) { // Traverse all nodes

        next = curr->next; // Save next node

        curr->next = prev; // Reverse current link

        prev = curr; // Move prev forward

        curr = next; // Move curr forward

    }

    return prev; // New head is prev

}

```

```

void printList(Node* head) {    // Print linked list

    Node* temp = head;        // Start from head

    while (temp != nullptr) {    // Until end

        cout << temp->data << " "; // Print data

        temp = temp->next;        // Move next

    }

    cout << endl;                // Newline

}

```

```

int main() {                    // Main function

    Node* head = nullptr;        // Empty list

    insertEnd(head, 1);          // Insert 1

    insertEnd(head, 2);          // Insert 2

    insertEnd(head, 3);          // Insert 3

    insertEnd(head, 4);          // Insert 4

    cout << "Original: ";        // Label

    printList(head);            // Print original

    head = reverseList(head);    // Reverse list

    cout << "Reversed: ";        // Label

    printList(head);            // Print reversed

    return 0;                    // End program

}

```

Output:

```
Original: 1 2 3 4
Reversed: 4 3 2 1

-----
Process exited after 0.1458 seconds with return value 0
Press any key to continue . . .
```

Task 03: Remove all duplicates contacts from singly linked list

```
#include <iostream>          // Include input-output library

using namespace std;        // Use standard namespace
```

```
struct Node {               // Node structure

    int data;               // Contact ID

    Node* next;            // Next pointer

};
```

```
Node* createNode(int value) { // Create node

    Node* n = new Node;      // Allocate memory

    n->data = value;         // Set data

    n->next = nullptr;       // Set next null

    return n;               // Return node

}
```

```
void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) {       // If empty

        head = n;               // Set head

    }
```

```

        return;          // Exit
    }

    Node* temp = head;    // Start from head
    while (temp->next != nullptr) { // Go to last
        temp = temp->next;    // Move forward
    }
    temp->next = n;        // Link last node
}

void removeDuplicates(Node* head) { // Remove duplicate values
    Node* current = head;    // Start from head
    while (current != nullptr && current->next != nullptr) { // Traverse each node
        Node* runner = current; // Runner starts from current
        while (runner->next != nullptr) { // Check all later nodes
            if (runner->next->data == current->data) { // If duplicate found
                Node* dup = runner->next; // Save duplicate node
                runner->next = runner->next->next; // Skip duplicate
                delete dup;    // Delete duplicate node
            } else {
                runner = runner->next; // Move runner forward
            }
        }
        current = current->next; // Move current forward
    }
}

```

```

void printList(Node* head) {    // Print list

    Node* temp = head;        // Start from head

    while (temp != nullptr) {    // Until end

        cout << temp->data << " "; // Print data

        temp = temp->next;        // Move next

    }

    cout << endl;                // Newline

}

int main() {                    // Main function

    Node* head = nullptr;        // Empty list

    insertEnd(head, 10);          // Insert 10

    insertEnd(head, 20);          // Insert 20

    insertEnd(head, 10);          // Insert duplicate 10

    insertEnd(head, 30);          // Insert 30

    insertEnd(head, 20);          // Insert duplicate 20

    cout << "Original: ";        // Label

    printList(head);              // Print original

    removeDuplicates(head);        // Remove duplicates

    cout << "After removing duplicates: "; // Label

    printList(head);              // Print final list

    return 0;                    // End program    }

```

Output:

```

Original: 10 20 10 30 20
After removing duplicates: 10 20 30
-----
Process exited after 0.1462 seconds with return value 0
Press any key to continue . . .

```

Task 04: Detect loop in singly linked list

```
#include <iostream>          // Include input-output library

using namespace std;        // Use standard namespace


struct Node {               // Node structure

    int data;               // Store data

    Node* next;            // Next pointer

};


Node* createNode(int value) { // Create node

    Node* n = new Node;     // Allocate memory

    n->data = value;         // Set data

    n->next = nullptr;       // Set next null

    return n;               // Return node

}


bool detectLoop(Node* head) { // Detect loop using Floyd cycle

    Node* slow = head;      // Slow pointer starts head

    Node* fast = head;      // Fast pointer starts head

    while (fast != nullptr && fast->next != nullptr) { // While fast can move

        slow = slow->next;    // Move slow by one

        fast = fast->next->next; // Move fast by two

        if (slow == fast) {   // If pointers meet

            return true;      // Loop exists

        }

    }

}
```

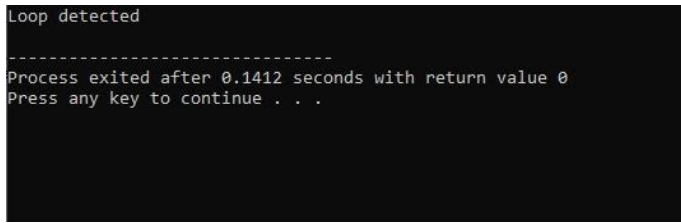
```

    return false;          // No loop found
}

int main() {              // Main function
    Node* head = createNode(1); // Create first node
    head->next = createNode(2); // Add second node
    head->next->next = createNode(3); // Add third node
    head->next->next->next = createNode(4); // Add fourth node
    head->next->next->next->next = head->next; // Create loop to node 2
    if (detectLoop(head)) { // Check loop
        cout << "Loop detected" << endl; // Print if loop
    } else {
        cout << "No loop" << endl; // Print if no loop
    }
    return 0;              // End program
}

```

Output:



```

Loop detected
-----
Process exited after 0.1412 seconds with return value 0
Press any key to continue . . .

```

Task 05: Find middle friend, left middle if even

```

#include <iostream>        // Include input-output library

using namespace std;      // Use standard namespace

struct Node {             // Node structure

```

```

    int data;           // Friend ID

    Node* next;         // Next pointer
};

Node* createNode(int value) {    // Create node

    Node* n = new Node;         // Allocate memory

    n->data = value;             // Set data

    n->next = nullptr;          // Set next null

    return n;                   // Return node
}

void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) {       // If empty

        head = n;                // Set head

        return;                  // Exit
    }

    Node* temp = head;           // Start from head

    while (temp->next != nullptr) { // Go to last

        temp = temp->next;        // Move forward
    }

    temp->next = n;               // Link last node
}

Node* findLeftMiddle(Node* head) { // Find left middle node

    if (head == nullptr) return nullptr; // Empty list

```

```

Node* slow = head;          // Slow pointer

Node* fast = head;          // Fast pointer

while (fast->next != nullptr && fast->next->next != nullptr) { // For left middle

    slow = slow->next;        // Move slow by one

    fast = fast->next->next;    // Move fast by two

}

return slow;                // Return middle node

}

int main() {                // Main function

    Node* head = nullptr;    // Empty list

    insertEnd(head, 1);       // Insert 1

    insertEnd(head, 2);       // Insert 2

    insertEnd(head, 3);       // Insert 3

    insertEnd(head, 4);       // Insert 4

    Node* middle = findLeftMiddle(head); // Find middle

    if (middle != nullptr) {   // If list not empty

        cout << "Left middle: " << middle->data << endl; // Print middle

    }

    return 0;                // End program

}

```

Output:

```

Left middle: 2
-----
Process exited after 0.1413 seconds with return value 0
Press any key to continue . . .

```

Task 06: Divide singly linked list into even-priced and odd-priced lines

```
#include <iostream>          // Include input-output library
using namespace std;         // Use standard namespace
```

```
struct Node {                // Node structure
    int data;                 // Fruit price
    Node* next;              // Next pointer
};
```

```
Node* createNode(int value) { // Create node
    Node* n = new Node;       // Allocate memory
    n->data = value;           // Set data
    n->next = nullptr;         // Set next null
    return n;                 // Return node
}
```

```
void insertEnd(Node*& head, int value) { // Insert at end
    Node* n = createNode(value); // Create node
    if (head == nullptr) {       // If empty
        head = n;                // Set head
        return;                  // Exit
    }
    Node* temp = head;           // Start from head
    while (temp->next != nullptr) { // Go to last
        temp = temp->next;       // Move forward
    }
```

```

temp->next = n;          // Link last node
}

void splitEvenOdd(Node* head, Node*& evenHead, Node*& oddHead) { // Split list

    evenHead = nullptr;    // Even list head
    oddHead = nullptr;     // Odd list head
    Node* evenTail = nullptr; // Even list tail
    Node* oddTail = nullptr;  // Odd list tail
    Node* curr = head;       // Start from head
    while (curr != nullptr) { // Traverse original list
        Node* nextNode = curr->next; // Save next node
        curr->next = nullptr; // Detach current node
        if (curr->data % 2 == 0) { // If price is even
            if (evenHead == nullptr) { // First even node
                evenHead = curr; // Set even head
                evenTail = curr; // Set even tail
            } else {
                evenTail->next = curr; // Link to even list
                evenTail = curr; // Update even tail
            }
        } else { // If price is odd
            if (oddHead == nullptr) { // First odd node
                oddHead = curr; // Set odd head
                oddTail = curr; // Set odd tail
            } else {
                oddTail->next = curr; // Link to odd list
            }
        }
    }
}

```

```

        oddTail = curr;    // Update odd tail
    }
}

curr = nextNode;    // Move to next node
}
}

void printList(Node* head) {    // Print list
    Node* temp = head;    // Start from head
    while (temp != nullptr) {    // Until end
        cout << temp->data << " "; // Print data
        temp = temp->next;    // Move next
    }
    cout << endl;    // Newline
}

int main() {    // Main function
    Node* head = nullptr;    // Empty list
    insertEnd(head, 10);    // Insert even
    insertEnd(head, 15);    // Insert odd
    insertEnd(head, 22);    // Insert even
    insertEnd(head, 7);    // Insert odd
    insertEnd(head, 8);    // Insert even

    Node* evenHead = nullptr;    // Even list head
    Node* oddHead = nullptr;    // Odd list head
    splitEvenOdd(head, evenHead, oddHead); // Split list

```

```

cout << "Even prices: ";    // Label

printList(evenHead);        // Print even list

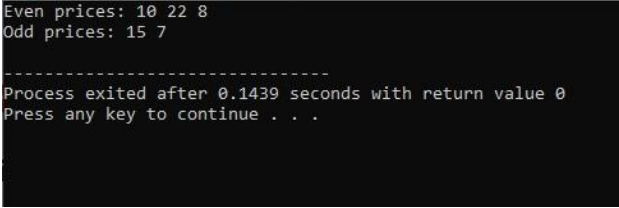
cout << "Odd prices: ";     // Label

printList(oddHead);         // Print odd list

return 0;                    // End program
}

```

Output:



```

Even prices: 10 22 8
Odd prices: 15 7

-----
Process exited after 0.1439 seconds with return value 0
Press any key to continue . . .

```

Task 07: Reverse first half and second half separately

```

#include <iostream>           // Include input-output library

using namespace std;         // Use standard namespace


struct Node {                // Node structure

    int data;                 // Store data

    Node* next;               // Next pointer

};


Node* createNode(int value) { // Create node

    Node* n = new Node;       // Allocate memory

    n->data = value;           // Set data

    n->next = nullptr;         // Set next null

    return n;                 // Return node

}

```

```
void insertEnd(Node*& head, int value) { // Insert at end
```

```
    Node* n = createNode(value); // Create node
```

```
    if (head == nullptr) { // If empty
```

```
        head = n; // Set head
```

```
        return; // Exit
```

```
    }
```

```
    Node* temp = head; // Start from head
```

```
    while (temp->next != nullptr) { // Go to last
```

```
        temp = temp->next; // Move forward
```

```
    }
```

```
    temp->next = n; // Link last node
```

```
}
```

```
Node* reverseNodes(Node* head) { // Reverse a list
```

```
    Node* prev = nullptr; // Previous pointer
```

```
    Node* curr = head; // Current pointer
```

```
    Node* next = nullptr; // Next pointer
```

```
    while (curr != nullptr) { // Traverse list
```

```
        next = curr->next; // Save next
```

```
        curr->next = prev; // Reverse link
```

```
        prev = curr; // Move prev
```

```
        curr = next; // Move curr
```

```
    }
```

```
    return prev; // Return new head
```

```
}
```

```

Node* reverseHalves(Node* head) { // Reverse both halves

    if (head == nullptr || head->next == nullptr) return head; // If 0 or 1 node

    int n = 0;                // Count nodes

    Node* temp = head;        // Start from head

    while (temp != nullptr) { // Count all nodes

        n++;                // Increase count

        temp = temp->next;    // Move next
    }

    int firstCount = n / 2;    // First half size

    Node* firstHead = head;    // First half head

    Node* firstTail = head;    // First half tail

    for (int i = 1; i < firstCount; i++) { // Find first half tail

        firstTail = firstTail->next; // Move forward
    }

    Node* secondHead = firstTail->next; // Second half head

    firstTail->next = nullptr; // Split list

    Node* newFirstHead = reverseNodes(firstHead); // Reverse first half

    Node* newSecondHead = reverseNodes(secondHead); // Reverse second half

    firstHead->next = newSecondHead; // Connect first half tail to second half

    return newFirstHead;        // Return new head
}

```

```

void printList(Node* head) { // Print list

    Node* temp = head;        // Start from head

    while (temp != nullptr) { // Until end

```

```

        cout << temp->data << " "; // Print data

        temp = temp->next;        // Move next
    }

    cout << endl;                // Newline
}

int main() {                    // Main function

    Node* head = nullptr;      // Empty list

    for (int i = 1; i <= 8; i++) { // Insert 1 to 8

        insertEnd(head, i);     // Insert current number
    }

    cout << "Original: ";      // Label

    printList(head);           // Print original

    head = reverseHalves(head); // Reverse halves

    cout << "After reversing halves: "; // Label

    printList(head);           // Print result

    return 0;                  // End program
}

```

Output:

```

Original: 1 2 3 4 5 6 7 8
After reversing halves: 4 3 2 1 8 7 6 5
-----
Process exited after 0.1409 seconds with return value 0
Press any key to continue . . .

```

Task 08: Remove duplicate stamps from singly linked list

```

#include <iostream>              // Include input-output library

using namespace std;           // Use standard namespace

```

```

struct Node {           // Node structure

    int data;           // Stamp design ID

    Node* next;         // Next pointer
};

```

```

Node* createNode(int value) {    // Create node

    Node* n = new Node;         // Allocate memory

    n->data = value;             // Set data

    n->next = nullptr;           // Set next null

    return n;                   // Return node
}

```

```

void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) {        // If empty

        head = n;                 // Set head

        return;                   // Exit
    }

    Node* temp = head;            // Start from head

    while (temp->next != nullptr) { // Go to last

        temp = temp->next;         // Move forward
    }

    temp->next = n;               // Link last node
}

```

```

void removeDuplicates(Node* head) { // Remove duplicate stamps

    Node* current = head;        // Start from head

    while (current != nullptr && current->next != nullptr) { // For each node

        Node* runner = current;  // Runner starts at current

        while (runner->next != nullptr) { // Check later nodes

            if (runner->next->data == current->data) { // If duplicate

                Node* dup = runner->next; // Save duplicate

                runner->next = runner->next->next; // Skip duplicate

                delete dup;           // Delete duplicate

            } else {

                runner = runner->next; // Move runner

            }

        }

        current = current->next; // Move current

    }

}

```

```

void printList(Node* head) { // Print list

    Node* temp = head;        // Start from head

    while (temp != nullptr) { // Until end

        cout << temp->data << " "; // Print data

        temp = temp->next;        // Move next

    }

    cout << endl;              // Newline

}

```

```

int main() {           // Main function

    Node* head = nullptr;    // Empty list

    insertEnd(head, 101);    // Insert stamp
    insertEnd(head, 102);    // Insert stamp
    insertEnd(head, 101);    // Insert duplicate
    insertEnd(head, 103);    // Insert stamp
    insertEnd(head, 102);    // Insert duplicate

    cout << "Original stamps: "; // Label
    printList(head);         // Print original

    removeDuplicates(head);   // Remove duplicates

    cout << "Unique stamps: "; // Label
    printList(head);         // Print final

    return 0;              // End program
}

```

Output:

```

Original stamps: 101 102 101 103 102
Unique stamps: 101 102 103

-----
Process exited after 0.15 seconds with return value 0
Press any key to continue . . .

```

Task 09: Delete all instances of a given book identifier

```

#include <iostream>      // Include input-output library

using namespace std;    // Use standard namespace


struct Node {           // Node structure

    int data;           // Book identifier

    Node* next;         // Next pointer
}

```

```
};
```

```
Node* createNode(int value) {    // Create node

    Node* n = new Node;          // Allocate memory

    n->data = value;              // Set data

    n->next = nullptr;           // Set next null

    return n;                    // Return node

}
```

```
void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) {        // If empty

        head = n;                 // Set head

        return;                   // Exit

    }

    Node* temp = head;            // Start from head

    while (temp->next != nullptr) { // Go to last

        temp = temp->next;        // Move forward

    }

    temp->next = n;               // Link last node

}
```

```
Node* deleteAll(Node* head, int key) { // Delete all matching nodes

    while (head != nullptr && head->data == key) { // If head matches

        Node* temp = head;        // Save head

        head = head->next;        // Move head forward

    }

}
```

```

        delete temp;          // Delete old head
    }

    Node* current = head;      // Start from new head
    while (current != nullptr && current->next != nullptr) { // Traverse
        if (current->next->data == key) { // If next matches
            Node* temp = current->next; // Save matching node
            current->next = current->next->next; // Skip it
            delete temp;          // Delete matching node
        } else {
            current = current->next; // Move current forward
        }
    }

    return head;              // Return updated head
}

```

```

void printList(Node* head) {    // Print list
    Node* temp = head;          // Start from head
    while (temp != nullptr) {    // Until end
        cout << temp->data << " "; // Print data
        temp = temp->next;        // Move next
    }
    cout << endl;              // Newline
}

```

```

int main() {                    // Main function
    Node* head = nullptr;       // Empty list

```

```

insertEnd(head, 3);      // Insert 3 at beginning
insertEnd(head, 5);      // Insert 5
insertEnd(head, 3);      // Insert 3 in middle
insertEnd(head, 7);      // Insert 7
insertEnd(head, 3);      // Insert 3 at end
cout << "Original: ";    // Label
printList(head);         // Print original
head = deleteAll(head, 3); // Delete all 3s
cout << "After deleting 3: "; // Label
printList(head);         // Print final
return 0;                // End program
}

```

Output:

```

Original: 3 5 3 7 3
After deleting 3: 5 7
-----
Process exited after 0.1395 seconds with return value 0
Press any key to continue . . .

```

Task 10: Swap every two consecutive nodes in singly linked list

```

#include <iostream>        // Include input-output library
using namespace std;      // Use standard namespace

struct Node {             // Node structure
    int data;              // Task ID
    Node* next;            // Next pointer
};

```

```

Node* createNode(int value) {    // Create node

    Node* n = new Node;        // Allocate memory

    n->data = value;            // Set data

    n->next = nullptr;          // Set next null

    return n;                  // Return node
}

```

```

void insertEnd(Node*& head, int value) { // Insert at end

    Node* n = createNode(value); // Create node

    if (head == nullptr) {        // If empty

        head = n;                // Set head

        return;                  // Exit

    }

    Node* temp = head;            // Start from head

    while (temp->next != nullptr) { // Go to last

        temp = temp->next;        // Move forward

    }

    temp->next = n;                // Link last node

}

```

```

Node* swapPairs(Node* head) {    // Swap every two nodes

    if (head == nullptr || head->next == nullptr) return head; // If less than 2 nodes

    Node* newHead = head->next;   // Second node becomes new head

    Node* prev = nullptr;        // Previous pair tail

    Node* curr = head;           // Current node

```

```

while (curr != nullptr && curr->next != nullptr) { // While pair exists

    Node* first = curr;      // First node of pair

    Node* second = curr->next; // Second node of pair

    Node* nextPair = second->next; // Next pair start

    second->next = first;      // Second points to first

    first->next = nextPair;    // First points to next pair

    if (prev != nullptr) {    // If previous pair exists

        prev->next = second;   // Connect previous pair to second

    }

    prev = first;             // First becomes previous tail

    curr = nextPair;          // Move to next pair

}

return newHead;              // Return new head

}

```

```

void printList(Node* head) {    // Print list

    Node* temp = head;          // Start from head

    while (temp != nullptr) {   // Until end

        cout << temp->data << " "; // Print data

        temp = temp->next;       // Move next

    }

    cout << endl;              // Newline

}

```

```

int main() {                    // Main function

    Node* head = nullptr;       // Empty list

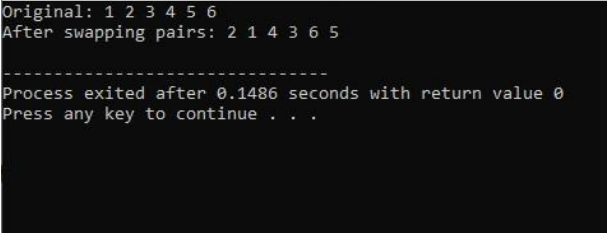
```

```

insertEnd(head, 1);      // Insert 1
insertEnd(head, 2);      // Insert 2
insertEnd(head, 3);      // Insert 3
insertEnd(head, 4);      // Insert 4
insertEnd(head, 5);      // Insert 5
insertEnd(head, 6);      // Insert 6
cout << "Original: ";    // Label
printList(head);         // Print original
head = swapPairs(head);  // Swap pairs
cout << "After swapping pairs: "; // Label
printList(head);         // Print result
return 0;                // End program
}

```

Output:



```

Original: 1 2 3 4 5 6
After swapping pairs: 2 1 4 3 6 5

-----
Process exited after 0.1486 seconds with return value 0
Press any key to continue . . .

```

Doubly Linked List

Task 01: Swap desks symmetrically toward center

```

#include <iostream>        // Include input-output stream
#include <string>           // Include string class
using namespace std;      // Use standard namespace

```

```

struct Node {           // Node structure for doubly linked list

    string name;         // Data: desk/person name

    Node* next;          // Pointer to next node

    Node* prev;          // Pointer to previous node

};                       // End of Node structure


Node* createNode(string name) { // Function to create a new node

    Node* newNode = new Node;   // Allocate memory for new node

    newNode->name = name;        // Set node data

    newNode->next = nullptr;     // Initialize next pointer to null

    newNode->prev = nullptr;     // Initialize prev pointer to null

    return newNode;             // Return new node

}                               // End of createNode


void insertAtEnd(Node*& head, string name) { // Insert node at end

    Node* newNode = createNode(name); // Create new node

    if (head == nullptr) {           // If list is empty

        head = newNode;              // New node becomes head

        return;                      // Exit function

    }                               // End if

    Node* temp = head;               // Start from head

    while (temp->next != nullptr) {   // Traverse to last node

        temp = temp->next;            // Move to next node

    }                               // End while

    temp->next = newNode;             // Link old last to new node

    newNode->prev = temp;             // Link new node back to old last

```

```
} // End insertAtEnd
```

```
void printForward(Node* head) { // Print list from head to tail

    Node* temp = head; // Start from head

    while (temp != nullptr) { // Until end of list

        cout << temp->name; // Print current name

        if (temp->next != nullptr) // If not last node

            cout << " <-> "; // Print separator

        temp = temp->next; // Move to next node

    } // End while

    cout << endl; // Print new line

} // End printForward
```

```
void swapSymmetric(Node* head) { // Swap first with last, second with second-last, etc.

    if (head == nullptr) return; // If list empty, do nothing

    Node* tail = head; // Start tail from head

    while (tail->next != nullptr) { // Traverse to last node

        tail = tail->next; // Move forward

    } // End while

    Node* left = head; // Left pointer starts at head

    Node* right = tail; // Right pointer starts at tail

    while (left != right && left->prev != right) { // Stop when pointers meet/cross

        string temp = left->name; // Store left data

        left->name = right->name; // Copy right data to left

        right->name = temp; // Copy stored data to right

        left = left->next; // Move left forward
```

```

        right = right->prev;          // Move right backward
    }                                // End while
}                                    // End swapSymmetric

int main() {                        // Main function
    Node* head = nullptr;          // Initialize empty list
    insertAtEnd(head, "Alice");     // Insert Alice
    insertAtEnd(head, "Bob");       // Insert Bob
    insertAtEnd(head, "Charlie");   // Insert Charlie
    insertAtEnd(head, "Dana");      // Insert Dana
    insertAtEnd(head, "Eva");       // Insert Eva
    insertAtEnd(head, "Frank");     // Insert Frank
    cout << "Original: ";          // Print label
    printForward(head);             // Display original list
    swapSymmetric(head);            // Perform symmetric swaps
    cout << "After swaps: ";       // Print label
    printForward(head);             // Display modified list
    return 0;                      // Return success
}

```

Output:

```

Original: Alice <-> Bob <-> Charlie <-> Dana <-> Eva <-> Frank
After swaps: Frank <-> Eva <-> Dana <-> Charlie <-> Bob <-> Alice
-----
Process exited after 0.151 seconds with return value 0
Press any key to continue . . .

```

Task 02: Music player playlist management

```

#include <iostream>                // Include input-output stream

```

```

#include <string>           // Include string class

using namespace std;       // Use standard namespace


struct Song {              // Node for a song

    string title;           // Song title

    Song* next;             // Pointer to next song

    Song* prev;            // Pointer to previous song

};                          // End Song structure


class Playlist {           // Playlist class using doubly linked list

private:                   // Private members

    Song* head;            // First song

    Song* tail;            // Last song

    Song* current;         // Currently playing song

public:                   // Public methods

    Playlist() {           // Constructor

        head = nullptr;    // Initially empty

        tail = nullptr;    // Initially empty

        current = nullptr; // No current song

    }                     // End constructor


    void addSong(string title) { // Add song at end of playlist

        Song* newSong = new Song; // Allocate new song node

        newSong->title = title; // Set song title

        newSong->next = nullptr; // Next is null

        newSong->prev = nullptr; // Prev is null
    }
}

```

```

if (head == nullptr) {    // If playlist empty

    head = newSong;        // Head becomes new song

    tail = newSong;        // Tail becomes new song

    current = newSong;     // Current becomes new song
} else {                  // If playlist not empty

    tail->next = newSong;   // Old tail points to new song

    newSong->prev = tail;   // New song points back to old tail

    tail = newSong;        // Update tail
}                          // End if-else
}                          // End addSong


void removeSong(string title) { // Remove song by title

    Song* temp = head;        // Start from head

    while (temp != nullptr && temp->title != title) { // Search for title

        temp = temp->next;    // Move to next song
    }                        // End while

    if (temp == nullptr) {    // If not found

        cout << "Song not found: " << title << endl; // Print message

        return;              // Exit method
    }                        // End if

    if (temp == head) {      // If removing head

        head = temp->next;    // Move head forward
    }                        // End if

    if (temp == tail) {      // If removing tail

        tail = temp->prev;    // Move tail backward
    }                        // End if
}

```

```

if (temp->prev != nullptr) { // If node has previous
    temp->prev->next = temp->next; // Link previous to next
}
// End if

if (temp->next != nullptr) { // If node has next
    temp->next->prev = temp->prev; // Link next to previous
}
// End if

if (current == temp) { // If removing current song
    current = (temp->next != nullptr) ? temp->next : head; // Update current
}
// End if

delete temp; // Free memory
}
// End removeSong

```

```

void displayForward() { // Display playlist from first to last
    Song* temp = head; // Start from head
    while (temp != nullptr) { // Until end
        cout << temp->title; // Print title
        if (temp->next != nullptr) cout << " <-> "; // Separator
        temp = temp->next; // Move next
    }
    // End while
    cout << endl; // New line
}
// End displayForward

```

```

void displayBackward() { // Display playlist from last to first
    Song* temp = tail; // Start from tail
    while (temp != nullptr) { // Until beginning
        cout << temp->title; // Print title
    }
}

```

```

        if (temp->prev != nullptr) cout << " <-> "; // Separator

        temp = temp->prev;    // Move prev
    }                        // End while

    cout << endl;           // New line
}                            // End displayBackward

```

```

void playNext() {           // Play next song

    if (current == nullptr) { // If no current song

        current = head;      // Set current to head

    } else if (current->next != nullptr) { // If next exists

        current = current->next; // Move to next

    } else {                // If at end

        current = head;      // Wrap around to head

    }                        // End if-else

    if (current != nullptr) // If current exists

        cout << "Playing: " << current->title << endl; // Display

}                            // End playNext

```

```

void playPrevious() {       // Play previous song

    if (current == nullptr) { // If no current song

        current = tail;      // Set current to tail

    } else if (current->prev != nullptr) { // If previous exists

        current = current->prev; // Move to previous

    } else {                // If at beginning

        current = tail;      // Wrap around to tail

    }                        // End if-else

```

```

        if (current != nullptr)    // If current exists

            cout << "Playing: " << current->title << endl; // Display

    }                                // End playPrevious
};                                // End Playlist class

```

```

int main() {                        // Main function

    Playlist pl;                    // Create playlist object

    pl.addSong("Song A");           // Add song A
    pl.addSong("Song B");           // Add song B
    pl.addSong("Song C");           // Add song C
    pl.addSong("Song D");           // Add song D

    cout << "Forward: ";           // Print label
    pl.displayForward();             // Display forward

    cout << "Backward: ";          // Print label
    pl.displayBackward();           // Display backward

    pl.playNext();                  // Play next
    pl.playNext();                  // Play next

    pl.playPrevious();              // Play previous

    pl.removeSong("Song B");         // Remove Song B

    cout << "After removal: ";      // Print label

    pl.displayForward();            // Display forward again

    return 0;                       // Return success
}

```

Output:

```
Forward: Song A <-> Song B <-> Song C <-> Song D
Backward: Song D <-> Song C <-> Song B <-> Song A
Playing: Song B
Playing: Song C
Playing: Song B
After removal: Song A <-> Song C <-> Song D

-----
Process exited after 0.1402 seconds with return value 0
Press any key to continue . . .
```

Task 03: Inventory management system

```
#include <iostream>           // Include input-output stream
```

```
#include <string>             // Include string class
```

```
using namespace std;         // Use standard namespace
```

```
struct Item {                // Node for an item

    string name;              // Item name

    string location;          // Location inside section

    Item* next;               // Pointer to next item

    Item* prev;               // Pointer to previous item

};                             // End Item structure
```

```
struct Section {             // Node for a section

    string name;              // Section name

    Item* itemHead;           // Head of items in this section

    Section* next;            // Pointer to next section

    Section* prev;            // Pointer to previous section

};                             // End Section structure
```

```
struct Store {               // Node for a store
```

```

string name;           // Store name

Section* sectionHead;  // Head of sections in this store

Store* next;           // Pointer to next store

Store* prev;           // Pointer to previous store
};                      // End Store structure

```

```

class Inventory {       // Inventory management class
private:                // Private members
    Store* storeHead;   // Head of stores list
public:                 // Public methods
    Inventory() {       // Constructor
        storeHead = nullptr; // Initially no stores
    }                  // End constructor
}

```

```

Store* findStore(string storeName) { // Find store by name

    Store* temp = storeHead; // Start from head

    while (temp != nullptr) { // Traverse stores

        if (temp->name == storeName) return temp; // Found

        temp = temp->next; // Move next
    }                      // End while

    return nullptr;        // Not found
}                          // End findStore

```

```

Section* findSection(Store* store, string sectionName) { // Find section in store

    if (store == nullptr) return nullptr; // Invalid store

    Section* temp = store->sectionHead; // Start from section head

```

```

while (temp != nullptr) {           // Traverse sections

    if (temp->name == sectionName) return temp; // Found

    temp = temp->next;               // Move next
}                                   // End while

return nullptr;                     // Not found
}                                   // End findSection

```

```

void addStore(string storeName) {    // Add a new store

    Store* newStore = new Store;    // Allocate new store

    newStore->name = storeName;      // Set store name

    newStore->sectionHead = nullptr; // No sections yet

    newStore->next = nullptr;        // Next null

    newStore->prev = nullptr;        // Prev null

    if (storeHead == nullptr) {     // If no stores

        storeHead = newStore;       // New store becomes head
    } else {                         // If stores exist

        Store* temp = storeHead;    // Start from head

        while (temp->next != nullptr) { // Go to last store

            temp = temp->next;        // Move next
        }                           // End while

        temp->next = newStore;        // Link last to new store

        newStore->prev = temp;        // Link new store back
    }                               // End if-else
}                                   // End addStore

```

```

void addSection(string storeName, string sectionName) { // Add section to store

```

```

Store* store = findStore(storeName); // Find store

if (store == nullptr) {           // If store not found

    cout << "Store not found\n";    // Print error

    return;                        // Exit
}                                  // End if

Section* newSection = new Section; // Allocate new section

newSection->name = sectionName;     // Set section name

newSection->itemHead = nullptr;     // No items yet

newSection->next = nullptr;         // Next null

newSection->prev = nullptr;         // Prev null

if (store->sectionHead == nullptr) { // If no sections

    store->sectionHead = newSection; // New section becomes head
} else {                            // If sections exist

    Section* temp = store->sectionHead; // Start from head

    while (temp->next != nullptr) {    // Go to last section

        temp = temp->next;             // Move next
    }                                  // End while

    temp->next = newSection;           // Link last to new

    newSection->prev = temp;           // Link new back
}                                     // End if-else

cout << "Section added.\n";          // Confirmation
}                                    // End addSection

void addItem(string storeName, string sectionName, string itemName, string location) { // Add item

    Store* store = findStore(storeName); // Find store

    if (store == nullptr) {           // If store not found

```

```

        cout << "Store not found\n";    // Print error

        return;                        // Exit
    }                                  // End if

    Section* section = findSection(store, sectionName); // Find section

    if (section == nullptr) {          // If section not found

        cout << "Section not found\n";  // Print error

        return;                        // Exit
    }                                  // End if

    Item* newItem = new Item;          // Allocate new item

    newItem->name = itemName;           // Set item name

    newItem->location = location;        // Set item location

    newItem->next = nullptr;             // Next null

    newItem->prev = nullptr;             // Prev null

    if (section->itemHead == nullptr) { // If no items

        section->itemHead = newItem;    // New item becomes head
    } else {                            // If items exist

        Item* temp = section->itemHead; // Start from head

        while (temp->next != nullptr) { // Go to last item

            temp = temp->next;           // Move next
        }                               // End while

        temp->next = newItem;            // Link last to new item

        newItem->prev = temp;            // Link new item back
    }                                   // End if-else

    cout << "Item added.\n";            // Confirmation
}                                     // End addItem

```

```

void removeItem(string storeName, string sectionName, string itemName) { // Remove item

    Store* store = findStore(storeName); // Find store

    if (store == nullptr) {          // If store not found

        cout << "Store not found\n"; // Print error

        return;                      // Exit
    }                                // End if

    Section* section = findSection(store, sectionName); // Find section

    if (section == nullptr) {        // If section not found

        cout << "Section not found\n"; // Print error

        return;                      // Exit
    }                                // End if

    Item* temp = section->itemHead; // Start from item head

    while (temp != nullptr && temp->name != itemName) { // Search item

        temp = temp->next;           // Move next
    }                                // End while

    if (temp == nullptr) {          // If not found

        cout << "Item not found\n"; // Print error

        return;                      // Exit
    }                                // End if

    if (temp == section->itemHead) { // If removing head item

        section->itemHead = temp->next; // Move head forward
    }                                // End if

    if (temp->prev != nullptr) {     // If has previous

        temp->prev->next = temp->next; // Link previous to next
    }                                // End if

    if (temp->next != nullptr) {     // If has next

```

```

        temp->next->prev = temp->prev;    // Link next to previous
    }                                    // End if

    delete temp;                        // Free memory

    cout << "Item removed.\n";          // Confirmation
}                                       // End removeItem


void displaySectionItems(string storeName, string sectionName) { // Display items in section

    Store* store = findStore(storeName); // Find store

    if (store == nullptr) {            // If store not found

        cout << "Store not found\n";    // Print error

        return;                        // Exit
    }                                    // End if

    Section* section = findSection(store, sectionName); // Find section

    if (section == nullptr) {          // If section not found

        cout << "Section not found\n";  // Print error

        return;                        // Exit
    }                                    // End if

    cout << "Items in " << storeName << " / " << sectionName << ":\n"; // Header

    Item* temp = section->itemHead;    // Start from item head

    while (temp != nullptr) {          // Traverse items

        cout << temp->name << " (Location: " << temp->location << ")\n"; // Print item

        temp = temp->next;              // Move next
    }                                    // End while
}                                       // End displaySectionItems


void displayStoreItems(string storeName) { // Display all items in a store

```

```

Store* store = findStore(storeName); // Find store

if (store == nullptr) {           // If store not found

    cout << "Store not found\n";    // Print error

    return;                        // Exit
}                                  // End if

cout << "Items in store " << storeName << ":\n"; // Header

Section* sec = store->sectionHead; // Start from section head

while (sec != nullptr) {          // Traverse sections

    cout << "Section: " << sec->name << "\n"; // Print section

    Item* item = sec->itemHead;    // Start from item head

    while (item != nullptr) {      // Traverse items

        cout << " " << item->name << " (Location: " << item->location << ")\n"; // Print item

        item = item->next;         // Move next
    }                              // End while

    sec = sec->next;               // Move next section
}                                  // End while
}                                  // End displayStoreItems
};                                 // End Inventory class


int main() {                      // Main function

    Inventory inv;                // Create inventory object

    inv.addStore("StoreA");        // Add store A

    inv.addSection("StoreA", "Toys"); // Add Toys section

    inv.addItem("StoreA", "Toys", "Car", "A1"); // Add item Car

    inv.addItem("StoreA", "Toys", "Doll", "A2"); // Add item Doll

    inv.addSection("StoreA", "Grocery"); // Add Grocery section

```

```

inv.addItem("StoreA", "Grocery", "Apple", "B1"); // Add item Apple

inv.displaySectionItems("StoreA", "Toys"); // Display Toys items

inv.displayStoreItems("StoreA");           // Display all StoreA items

inv.removeItem("StoreA", "Toys", "Car"); // Remove Car

inv.displaySectionItems("StoreA", "Toys"); // Display Toys again

return 0;                                // Return success
}

```

Output:

```

Section added.
Item added.
Item added.
Section added.
Item added.
Items in StoreA / Toys:
Car (Location: A1)
Doll (Location: A2)
Items in store StoreA:
Section: Toys
    Car (Location: A1)
    Doll (Location: A2)
Section: Grocery
    Apple (Location: B1)
Item removed.
Items in StoreA / Toys:
Doll (Location: A2)

-----
Process exited after 0.158 seconds with return value 0
Press any key to continue . . .

```

Task 04: Theater seating pattern

```

#include <iostream>           // Include input-output stream

using namespace std;        // Use standard namespace


struct Node {                // Node structure for doubly linked list

    int data;                // Seat number

    Node* next;              // Pointer to next node

    Node* prev;              // Pointer to previous node

};                            // End Node structure


Node* createNode(int data) { // Create a new node

```

```

Node* newNode = new Node;    // Allocate memory

newNode->data = data;        // Set data

newNode->next = nullptr;     // Next null

newNode->prev = nullptr;     // Prev null

return newNode;             // Return new node
}                             // End createNode


void insertAtEnd(Node*& head, int data) { // Insert node at end

    Node* newNode = createNode(data);  // Create new node

    if (head == nullptr) {             // If list empty

        head = newNode;                // New node becomes head

        return;                        // Exit
    }                                   // End if

    Node* temp = head;                 // Start from head

    while (temp->next != nullptr) {     // Go to last node

        temp = temp->next;              // Move next
    }                                   // End while

    temp->next = newNode;               // Link old last to new

    newNode->prev = temp;               // Link new back to old last
}                                     // End insertAtEnd


void printForward(Node* head) {        // Print list forward

    Node* temp = head;                 // Start from head

    while (temp != nullptr) {          // Until end

        cout << temp->data;             // Print data

        if (temp->next != nullptr)     // If not last

```

```

        cout << " -> ";          // Print arrow

        temp = temp->next;          // Move next
    }                               // End while

    cout << " -> NULL" << endl;    // Print end
}                                   // End printForward


void alternateSwap(Node* head) {    // Swap seats alternately from both ends

    if (head == nullptr) return;    // If empty, do nothing

    int n = 1;                      // Count nodes, start with 1

    Node* tail = head;              // Tail pointer

    while (tail->next != nullptr) {  // Traverse to last node

        tail = tail->next;           // Move forward

        n++;                        // Increment count
    }                               // End while

    if (n < 3) return;              // Need at least 3 nodes

    Node* left = head->next;         // Left starts at second node

    Node* right = tail->prev;        // Right starts at second-last node

    for (int l = 2, r = n - 1; l < r; l += 2, r -= 2) { // Swap pairs

        int temp = left->data;       // Store left data

        left->data = right->data;     // Copy right to left

        right->data = temp;          // Copy temp to right

        if (left->next != nullptr && left->next->next != nullptr) // If can move two steps

            left = left->next->next;  // Move left forward two

        else

            left = nullptr;          // Otherwise null

        if (right->prev != nullptr && right->prev->prev != nullptr) // If can move two steps

```

```

        right = right->prev->prev; // Move right backward two
    else
        right = nullptr; // Otherwise null
    } // End for
} // End alternateSwap

```

```

int main() { // Main function
    Node* head = nullptr; // Initialize empty list
    for (int i = 1; i <= 9; i++) { // Insert 1 to 9
        insertAtEnd(head, i); // Insert each number
    } // End for
    cout << "Original: "; // Print label
    printForward(head); // Display original
    alternateSwap(head); // Perform alternate swaps
    cout << "After calling method: "; // Print label
    printForward(head); // Display modified
    return 0; // Return success
}

```

Output:

```

Original: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> NULL
After calling method: 1 -> 8 -> 3 -> 6 -> 5 -> 4 -> 7 -> 2 -> 9 -> NULL
-----
Process exited after 0.1482 seconds with return value 0
Press any key to continue . . .

```

Circular Linked List

- **Josephus Problem using circular linked list**

```
#include <iostream>          // Include input-output stream library.

using namespace std;        // Use standard namespace.


struct Node {               // Define a node for circular linked list.

    int data;               // Store person number.

    Node* next;            // Pointer to next node.

};                          // End of Node structure.


Node* createCircle(int n) {  // Function to create circular linked list of 1..n.

    if (n <= 0) return nullptr; // Return null if invalid count.

    Node* head = new Node;    // Allocate first node.

    head->data = 1;           // Set first person number.

    head->next = nullptr;     // Initialize next pointer.

    Node* last = head;       // last points to last created node.

    for (int i = 2; i <= n; i++) { // Create persons 2 to n.

        Node* newNode = new Node; // Allocate new node.

        newNode->data = i;        // Set person number.

        newNode->next = nullptr;  // Initialize next pointer.

        last->next = newNode;     // Link old last to new node.

        last = newNode;          // Update last.

    }                            // End for.

    last->next = head;           // Make list circular.

    return head;                // Return head of circle.

}                              // End createCircle.
```

```

int josephus(int n, int m) {    // Function returns survivor position.

    if (n <= 0 || m <= 0) return -1; // Validate inputs.

    Node* head = createCircle(n); // Create circle.

    Node* curr = head;           // curr starts at head.

    Node* prev = head;           // prev will point to node before curr.

    while (prev->next != head) { // Find last node.

        prev = prev->next;       // Move prev forward.
    }                             // Now prev is last node.

    while (curr->next != curr) { // Loop until one node remains.

        for (int i = 1; i < m; i++) { // Skip m-1 persons.

            prev = curr;           // Move prev to curr.

            curr = curr->next;      // Move curr to next person.

        }                         // End for; curr is Mth person to eliminate.

        prev->next = curr->next;    // Unlink curr from circle.

        if (curr == head) {       // If head is being deleted,

            head = curr->next;     // Update head.

        }                         // End if.

        Node* temp = curr;        // Store node to delete.

        curr = prev->next;         // Move curr to next person.

        delete temp;              // Free memory of eliminated person.

    }                             // End while.

    int survivor = curr->data;     // Store survivor number.

    delete curr;                 // Delete last node.

    return survivor;              // Return survivor.

}                                // End josephus.

```

```

int main() {           // Program starts here.

    int n, m;           // n = total persons, m = count.

    cout << "Enter number of persons N: "; // Prompt for N.

    cin >> n;           // Read N.

    cout << "Enter count M: "; // Prompt for M.

    cin >> m;           // Read M.

    int result = josephus(n, m); // Call Josephus function.

    if (result == -1) { // Check invalid input.

        cout << "Invalid input.\n"; // Print error.

    } else {           // Otherwise,

        cout << "Safe position is: " << result << endl; // Print survivor.

    }                 // End if.

    return 0;         // Return success.

}

```

Output:

```

Enter number of persons N: 3
Enter count M: 4
Safe position is: 2

-----
Process exited after 6.533 seconds with return value 0
Press any key to continue . . .

```

Task 01: Task Scheduling system using circular linked list

```

#include <iostream>      // For input/output

#include <string>        // For string type

using namespace std;    // Use standard namespace

```

```
// Node for circular linked list
```

```
struct Task {  
    string name;           // Task name  
    int priority;          // Task priority  
    string status;         // pending, in-progress, completed  
    Task* next;            // Pointer to next task
```

```
  
    Task(string n, int p, string s) { // Constructor  
        name = n;  
        priority = p;  
        status = s;  
        next = nullptr;  
    }  
};
```

```
class TaskList {  
private:  
    Task* head;            // Head of circular list  
    Task* current;         // Points to last accessed task for round-robin
```

```
public:  
  
    TaskList() {           // Constructor  
        head = nullptr;  
        current = nullptr;  
    }
```

```

// Add a new task at the end

void addTask(string name, int priority, string status) {

    Task* newNode = new Task(name, priority, status);

    if (head == nullptr) {    // If list is empty

        head = newNode;

        newNode->next = head; // Point to itself

        current = head;

    } else {

        Task* temp = head;

        while (temp->next != head) { // Find last node

            temp = temp->next;

        }

        temp->next = newNode;

        newNode->next = head;

    }

    cout << "Task " << name << " added.\n";

}

```

```

// Remove a task by name

void removeTask(string name) {

    if (head == nullptr) {

        cout << "List is empty.\n";

        return;

    }

    // If head is to be removed

    if (head->name == name) {

```

```

if (head->next == head) { // Only one node

    delete head;

    head = nullptr;

    current = nullptr;

} else {

    Task* last = head;

    while (last->next != head) {

        last = last->next;

    }

    last->next = head->next;

    Task* temp = head;

    head = head->next;

    if (current == temp) current = head;

    delete temp;

}

cout << "Task " << name << " removed.\n";

return;

}

// Search in rest of the list

Task* prev = head;

Task* curr = head->next;

while (curr != head) {

    if (curr->name == name) {

        prev->next = curr->next;

        if (current == curr) current = curr->next;

        delete curr;

```

```

        cout << "Task " << name << " removed.\n";

        return;
    }

    prev = curr;

    curr = curr->next;

}

cout << "Task " << name << " not found.\n";

}

```

// Get next pending task in round-robin order

```

void getNextTask() {
    if (head == nullptr) {
        cout << "No tasks in the list.\n";

        return;
    }

    if (current == nullptr) current = head;

    Task* start = current;

    do {
        if (current->status == "pending") {
            cout << "Next task: " << current->name
                << " (Priority: " << current->priority << ")\n";

            current = current->next; // Move for next call

            return;
        }

        current = current->next;
    } while (current != start);
}

```

```

    cout << "No pending task found.\n";
}

// Display all tasks in a simple format
void displayAll() {
    if (head == nullptr) {
        cout << "No tasks to display.\n";
        return;
    }
    cout << "\n--- Task List ---\n";
    Task* temp = head;
    do {
        cout << temp->name << " (Priority: " << temp->priority
            << ") - " << temp->status << "\n";
        temp = temp->next;
    } while (temp != head);
    cout << "-----\n";
}

// Update status of a task by name
void updateStatus(string name, string newStatus) {
    if (head == nullptr) {
        cout << "List is empty.\n";
        return;
    }
    Task* temp = head;

```

```

do {
    if (temp->name == name) {
        temp->status = newStatus;

        cout << "Status of " << name << " updated to "
            << newStatus << ".\n";

        return;
    }

    temp = temp->next;
} while (temp != head);

cout << "Task " << name << " not found.\n";
}
};

```

```

int main() {
    TaskList list;

    int choice;

    do {
        cout << "\n=== Task Scheduler ===\n";

        cout << "1. Add Task\n";

        cout << "2. Remove Task\n";

        cout << "3. Get Next Task\n";

        cout << "4. Display All Tasks\n";

        cout << "5. Update Task Status\n";

        cout << "6. Exit\n";

        cout << "Enter choice: ";

        cin >> choice;
    }
}

```

```
cin.ignore(); // Clear newline
```

```
switch (choice) {  
    case 1: {  
        string name, status;  
        int priority;  
        cout << "Enter task name: ";  
        getline(cin, name);  
        cout << "Enter priority: ";  
        cin >> priority;  
        cin.ignore();  
        cout << "Enter status (pending/in-progress/completed): ";  
        getline(cin, status);  
        list.addTask(name, priority, status);  
        break;  
    }  
    case 2: {  
        string name;  
        cout << "Enter task name to remove: ";  
        getline(cin, name);  
        list.removeTask(name);  
        break;  
    }  
    case 3: {  
        list.getNextTask();  
        break;  
    }  
}
```

```
    }  
    case 4: {  
        list.displayAll();  
        break;  
    }  
    case 5: {  
        string name, newStatus;  
        cout << "Enter task name to update: ";  
        getline(cin, name);  
        cout << "Enter new status: ";  
        getline(cin, newStatus);  
        list.updateStatus(name, newStatus);  
        break;  
    }  
    case 6: {  
        cout << "Exiting...\n";  
        break;  
    }  
    default: {  
        cout << "Invalid choice.\n";  
    }  
}  
} while (choice != 6);  
  
return 0;  
}
```

Output:

```
==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 1
Enter task name: Design
Enter priority: 1
Enter status (pending/in-progress/completed): pending
Task 'Design' added.

==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 1
Enter task name: Code
Enter priority: 2
Enter status (pending/in-progress/completed): in-progress
Task 'Code' added.

==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 3
Next task: Design (Priority: 1)

==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 5
Enter task name to update: Design
Enter new status: completed
Status of 'Design' updated to completed.

==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 2
Enter task name to remove: Code
Task 'Code' removed.

==== Task Scheduler ====
1. Add Task
2. Remove Task
3. Get Next Task
4. Display All Tasks
5. Update Task Status
6. Exit
Enter choice: 6
Exiting...

-----
Process exited after 88.93 seconds with return value 0
Press any key to continue . . .
```